gemini.google.com /app/11e19e65dd76fc70

Conversación con Gemini

Has dicho
Tu Nombre jueves 11 junio 2026 17:26

[root@server2asir remote]$ss -ulnp | grep 514

UNCONN 0 0 0.0.0.0:514 0.0.0.0:* users:(("rsyslogd",pid=761,fd=5))

UNCONN 0 0 [::]:514 [::]:* users:(("rsyslogd",pid=761,fd=6))

Tu Nombre jueves 11 junio 2026 17:26

[root@server2asir remote]$

Gemini ha dicho

¡Perfecto! El comando `ss` nos confirma al 100% que **`rsyslogd` está escuchando correctamente en el puerto UDP 514** de tu servidor central (`0.0.0.0:514`).

Si el puerto está abierto pero la carpeta con la IP de tu Windows (`10.2.7.66`) no se crea en `/var/log/remote/`, solo quedan dos opciones: o el Firewall de Linux está bloqueando los paquetes antes de que lleguen a Rsyslog, o el cliente Windows no está enviando nada.

Vamos a descartar el firewall y a forzar al cliente en un momento.

Paso 1: Asegurar el Firewall en el servidor central (`.99`)

Como tu servidor parece ser un Red Hat / Rocky Linux / CentOS (por el prompt de `root`), el cortafuegos por defecto es `firewalld`. Abre el puerto UDP de forma definitiva ejecutando esto en tu terminal:

Bash

```
firewall-cmd --add-port=514/udp --permanent
firewall-cmd --reload
```

(Si por un casual usaras Ubuntu en este server, el comando sería `ufw allow 514/udp`).

Paso 2: El truco para forzar a NXLog en Windows

Por defecto, el módulo `im_msvistalog` de NXLog solo envía los logs **nuevos** que se generan a partir del momento en que se inicia el servicio. Si tu Windows está tranquilo y no está pasando nada, no enviará nada y la carpeta en Linux seguirá vacía.

Vamos a obligar a Windows a generar un log de prueba para ver si viaja por la red:

1. Abre una consola de **PowerShell como Administrador** en tu máquina Windows.
2. Ejecuta este comando exacto para meter un log falso en el Visor de Eventos (System):

   PowerShell

   ```
   Write-EventLog -LogName System -Source "Service Control Manager" -EventID 7036 -EntryType Information -Message "Prueba de envio Syslog hacia el servidor central"
   ```

3. Regresa corriendo a la terminal de Linux y comprueba si la carpeta ya se ha creado:

   Bash

   ```
   ls -l /var/log/remote/
   ```

Paso 3: ¿Sigue sin aparecer? Revisa los errores de NXLog

Si tras el paso anterior la carpeta sigue sin existir, el problema está en el propio Windows, que tiene el servicio bloqueado o mal configurado internamente.

1. En el cliente Windows, ve a la carpeta: `C:\Program Files\nxlog\data\`
2. Abre el archivo llamado **`nxlog.log`** con el bloc de notas.
3. Vete al final del archivo y busca si hay líneas de color o que contengan la palabra **`ERROR`** o **`WARNING`**.

> 💡 **Errores típicos de NXLog:**
>
> - `INFO nxlog-ce-X.X.X started`: Significa que está bien.
> - `failed to open Windows Event Log...`: Problema de permisos de Windows (asegúrate de que el servicio se ejecuta como *SYSTEM* o *Administrador*).
> - `connecting to 10.2.7.99:514 failed`: Problema de red o de enrutamiento en el switch/red local.

Has dicho
.99 es un ubuntu server

Gemini ha dicho

¡Excelente aclaración! Al ser un **Ubuntu Server**, el cortafuegos por defecto no es `firewalld`, sino `ufw` (o en algunos entornos de aula/laboratorio, las reglas nativas de `iptables`).

Si `rsyslogd` está escuchando correctamente (como vimos con el comando `ss`) pero la carpeta `/var/log/remote/10.2.7.66` no se crea, el tráfico UDP se está perdiendo en el camino o el propio `rsyslog` de Ubuntu no sabe cómo interpretar los logs que le manda Windows.

Vamos a tiro fijo con estos 3 pasos en tu Ubuntu Server:

Paso 1: Asegurar la apertura en UFW (Ubuntu)

Ejecuta estos comandos como `root` para garantizar que el sistema operativo permita la entrada del puerto 514/UDP:

Bash

```
ufw allow 514/udp
ufw reload
```

Si el comando te devuelve un error estilo *"ufw command not found"* o *"is not active"*, significa que el firewall de Ubuntu está desactivado y el tráfico pasa libre, por lo que podemos saltar directamente al siguiente paso.

Paso 2: Verificar la plantilla dinámica de Rsyslog

Para que Rsyslog cree automáticamente carpetas con las direcciones IP (como las de tu captura anterior `10.2.7.3`, etc.), necesita tener una regla de **Template** activa.

Abre el archivo de configuración de rsyslog en el servidor:

Bash

```
nano /etc/rsyslog.conf
# O si las reglas están separadas:
nano /etc/rsyslog.d/50-default.conf
```

Busca las líneas que controlan los logs remotos. Para que funcione bien con tu estructura de carpetas, debería haber un bloque parecido a este:

Plaintext

```
$template RemoteLogs,"/var/log/remote/%FROMHOST-IP%/%PROGRAMNAME%.log"
*.* ?RemoteLogs
& stop
```

(Si has hecho cambios aquí, guarda con `Ctrl+O` y sal con `Ctrl+X`).

Reinicia el servicio para aplicar cualquier regla de filtrado:

Bash

```
systemctl restart rsyslog
```

Paso 3: El diagnóstico definitivo (tcpdump)

Si tras abrir el puerto y reiniciar rsyslog sigues sin ver la carpeta, vamos a comprobar si el cliente Windows realmente le está enviando paquetes a tu Ubuntu.

Instala y ejecuta esta herramienta en la terminal de Ubuntu:

Bash

```
apt-get install tcpdump -y   # Por si no está instalado
tcpdump -i any udp port 514 -nn
```

Deja ese comando corriendo en la pantalla. Verás que se queda esperando. Ahora ve a tu Windows y haz algo que obligue a enviar logs (reinicia el servicio NXLog desde la consola de `services.msc` de Windows, o ejecuta el comando de PowerShell para inyectar un log simulado).

¿Aparecen líneas en la terminal de Ubuntu al hacer esto?

- **SÍ:** Si ves tráfico con la IP `10.2.7.66`, la red está perfecta. El problema es que Rsyslog no reconoce el formato del log de NXLog (habría que revisar el archivo `nxlog.conf` en Windows para asegurarte de que use el formato `IETF` o `BSD` estándar de syslog).
- **NO:** La pantalla se queda completamente congelada. El problema está al 100% en Windows. NXLog está apagado, da error en su archivo `C:\Program Files\nxlog\data\nxlog.log` o el firewall interno de Windows impide la salida de datos por red.

Has dicho

[root@server2asir remote]$cat /etc/rsyslog.conf

/etc/rsyslog.conf configuration file for rsyslog

#

For more information install rsyslog-doc and see

/usr/share/doc/rsyslog-doc/html/configuration/index.html

#

Default logging rules can be found in /etc/rsyslog.d/50-default.conf

##################

MODULES

##################

module(load="imuxsock") # provides support for local system logging

#module(load="immark") # provides --MARK-- message capability

Esta parte ya venía configurada

Activar módulos de escucha UDP y TCP

provides UDP syslog reception

module(load="imudp")

input(type="imudp" port="514")

```
# Esto hubo que descomentarlo

# provides TCP syslog reception

module(load="imtcp")

input(type="imtcp" port="514")


# provides kernel logging support and enable non-kernel klog messages

module(load="imklog" permitnonkernelfacility="on")

## REGLAS PARA LOS LOG -- DIFERENCIAR POR TIPO DE LOG Y SI ES CRITICO


# 1. FILTRO DE ALERTAS TOTALES (Aquí interceptamos TODO lo grave de todos los equipos)

#if ($syslogseverity <= 3 or $msg contains "level=ERROR" or $msg contains "GRAFANA_TEST") then {

#    action(type="omfile" file="/var/log/ALERTA_TOTAL.log")

#    stop

#}


# 1. FILTRO DE ALERTAS TOTALES (Aquí interceptamos TODO lo grave de todos los equipos)

# Se ha añadido opciones para pillar los tipo Test y también para los mensajes de Mikrotik, que son diferentes

# El syslogseverity 3 pilla los crit, alert y emerg

if ($syslogseverity <= 3 or $msg contains "level=ERROR" or $msg contains "GRAFANA_TEST" or $programname contains "critical" or $msg contains "critical") then {

    action(type="omfile" file="/var/log/ALERTA_TOTAL.log")

    stop

}
```

```
# 2. Segregación de Base de Datos
if $programname == 'mysqld' or $msg contains 'mariadb' or $msg contains 'mysql' then {
    action(type="omfile" file="/var/log/BBDD_LOG.log")
    stop
}

# 3. Todo lo demás que sea remoto va a las carpetas dinámicas
if $fromhost-ip != '127.0.0.1' then {
    $template DynamicFiles,"/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log"
    *.* ?DynamicFiles
    stop
}
#############################
#### GLOBAL DIRECTIVES ####
#############################

#
# Use traditional timestamp format.
# To enable high precision timestamps, comment out the following line.
#
$ActionFileDefaultTemplate RSYSLOG_TraditionalFileFormat

# Filter duplicated messages
$RepeatedMsgReduction on

#
```

```
# Set the default permissions for all log files.
#
$FileOwner syslog
$FileGroup adm
$FileCreateMode 0640
$DirCreateMode 0755
$Umask 0022
$PrivDropToUser syslog
$PrivDropToGroup syslog


#
# Where to place spool and state files
#
$WorkDirectory /var/spool/rsyslog


#
# Include all config files in /etc/rsyslog.d/
#
$IncludeConfig /etc/rsyslog.d/*.conf
#añade esta línea para que guarde los logs de FOG en un archivo separado:
#if $fromhost-ip == '10.2.7.5' then /var/log/fog_remote.log
#& stop
 Tu Nombre  jueves 11 junio 2026 17:29
[root@server2asir remote]$
```

Gemini ha dicho

¡Impresionante la estructura de reglas que tienes montada en tu `rsyslog.conf`! El archivo está muy bien organizado, pero acabamos de encontrar el motivo exacto de por qué la carpeta del Windows no aparece por ningún lado.

Hay un choque lógico entre la **Regla 1** y la **Regla 3** de tu archivo. Vamos a analizar por qué se están "tragando" los logs de tu Windows:

El Problema: El filtro de Alertas Totales se come tus logs

Mira detenidamente lo que hace tu regla número 1:

Plaintext

```
if ($syslogseverity <= 3 or $msg contains "level=ERROR" or $msg contains
"GRAFANA_TEST" or $programname contains "critical" or $msg contains "critical") then
{
    action(type="omfile" file="/var/log/ALERTA_TOTAL.log")
    stop
}
```

¿Qué pasa con **NXLog (Windows)**? Windows es un sistema operativo muy ruidoso. En el momento en que arranca el servicio, o cuando lee el *Event Viewer*, casi siempre arrastra algún evento antiguo o nuevo catalogado como `Error`, `Critical` o con severidad alta (menor o igual a 3).

Como tu regla tiene un **`stop`** al final, en cuanto un log de Windows cumple **una sola** de esas condiciones (por ejemplo, contener la palabra "critical" o "level=ERROR"):

1. Rsyslog lo mete de cabeza en `/var/log/ALERTA_TOTAL.log`.
2. Ejecuta el `stop`.
3. **El log muere ahí.** Jamás llega a la Regla 3, que es la que se encarga de crear la carpeta dinámica `%HOSTNAME%` en `/var/log/remote/`.

¿Cómo comprobar si esto es lo que está pasando?

Es muy fácil de verificar. Ejecuta este comando en la terminal de tu Ubuntu para ver si los logs de la IP de Windows (`10.2.7.66`) se están quedando atrapados en el archivo de alertas:

Bash

```
grep "10.2.7.66" /var/log/ALERTA_TOTAL.log
# O si rsyslog traduce los nombres:
grep -i "windows" /var/log/ALERTA_TOTAL.log
```

Si al ejecutar esto te aparecen líneas de texto, ¡misterio resuelto! Los logs de Windows sí están llegando a Ubuntu, pero se los está quedando en exclusiva el archivo `ALERTA_TOTAL.log`.

La Solución: ¿Cómo hacer que cree la carpeta y también guarde la alerta?

Si quieres que las alertas sigan guardándose en `ALERTA_TOTAL.log` pero **sin impedir** que se cree la carpeta dinámica de la máquina para el resto de logs, tienes que quitar el `stop` de la Regla 1 **específicamente para las máquinas remotas**, o mover la regla dinámica arriba.

La forma más limpia y elegante de solucionarlo es cambiar el orden: **Primero procesamos la carpeta dinámica y luego las alertas locales**, o simplemente quitar el comando `stop` de la sección de alertas remota.

Sin embargo, para no romper tu lógica actual, haz la prueba del `grep`. Si ves que los logs están ahí, la comunicación red está perfecta y tu cliente Windows con NXLog funciona de diez; solo es cuestión de ajustar qué filtros llevan el comando `stop` y cuáles permiten que el log siga su camino hacia abajo.

Has dicho
Tu Nombre jueves 11 junio 2026 17:29

[root@server2asir remote]$grep "10.2.7.66" /var/log/ALERTA_TOTAL.log

Tu Nombre jueves 11 junio 2026 17:30

[root@server2asir remote]$grep -i "windows" /var/log/ALERTA_TOTAL.log

Tu Nombre jueves 11 junio 2026 17:31

[root@server2asir remote]$

Gemini ha dicho

Vaya, el hecho de que ambos comandos `grep` hayan devuelto un resultado completamente vacío (es decir, volvieron al prompt sin pintar texto) significa que los logs de Windows **tampoco** se están quedando atrapados en el archivo de alertas.

Esto nos da una certeza absoluta: **los logs de tu máquina Windows (10.2.7.66) no están llegando físicamente al servidor Ubuntu (`.99`)**. Si estuviesen entrando, se habrían guardado en la carpeta remota o se habrían desviado al archivo de alertas.

Dado que `rsyslog` funciona genial para el resto de equipos, el problema está 100% en el lado del **cliente Windows**.

Vamos a solucionarlo revisando los dos únicos puntos de fallo posibles en el cliente:

Paso 1: Ver el archivo de fallos de NXLog en Windows

NXLog guarda un diario interno donde apunta exactamente por qué no puede enviar logs o si tiene problemas de comunicación.

1. En tu máquina Windows, ve a la ruta: `C:\Program Files\nxlog\data\` (o donde lo tengas instalado).
2. Abre el archivo **`nxlog.log`** con el Bloc de notas.
3. Desplázate hasta el final de todo y lee las últimas líneas:
    - **¿Ves algún error de red?** Si dice algo como `connecting to 10.2.7.99:514 failed` o `reconnecting...`, es que el Windows no puede alcanzar al Ubuntu. Puede que el Firewall de salida de Windows Defender esté bloqueando a NXLog.
    - **¿Ves un error de permisos?** Si dice `failed to open Windows Event Log`, el servicio NXLog no tiene permisos de administrador para leer el Visor de Eventos de Windows.

Paso 2: Verificar el archivo `nxlog.conf`

Asegúrate de que en la configuración de NXLog en Windows hayas indicado correctamente la IP del Ubuntu y el protocolo UDP. El bloque de salida (`<Output>`) de tu archivo `C:\Program Files\nxlog\conf\nxlog.conf` debe lucir exactamente así:

Plaintext

```
<Output out_syslog>
    Module      om_udp
    Host        10.2.7.99
    Port        514
    Exec        to_syslog_ietf();
</Output>
```

> ⚠️ **Nota clave:** Revisa si en tu archivo de Windows pusiste `om_udp` (para UDP) o `om_tcp` (para TCP). Si pusiste `om_tcp`, recuerda que tu regla número 3 de Rsyslog en Linux busca `if $fromhost-ip != '127.0.0.1'`, pero a veces TCP requiere configuraciones de filtrado distintas en Rsyslog o el firewall bloquea el puerto TCP 514. **Lo estándar y más fácil para clase es usar `om_udp`.**

Si haces algún cambio en el archivo de texto, acuérdate de abrir la aplicación **Servicios** de Windows (`services.msc`), buscar el servicio **nxlog** y darle a **Reiniciar**.

Paso 3: El descarte definitivo con `tcpdump`

Para asegurarnos al 100% de si Windows llega a escupir algún paquete hacia la red, ejecuta esto en la terminal de tu **Ubuntu Server**:

Bash

```
tcpdump -i any udp port 514 and host 10.2.7.66 -nn
```

Deja el comando corriendo (la pantalla se quedará congelada). Acto seguido, ve a Windows y **reinicia el servicio NXLog**.

- Si la pantalla de Ubuntu **sigue completamente vacía**, Windows no está enviando nada (revisa el Firewall de Windows o la IP del archivo `.conf`).
- Si de repente **empiezan a salir líneas**, ¡bingo! Significa que el tráfico ya llega y rsyslog creará la carpeta inmediatamente.

Has dicho
2026-05-22 17:09:21 WARNING no functional input modules!

2026-05-22 17:09:21 WARNING no routes defined!

2026-05-22 17:09:21 INFO nxlog-ce-3.2.2329 started

2026-05-22 17:12:43 WARNING stopping nxlog service

2026-05-22 17:12:43 WARNING nxlog-ce received a termination request signal, exiting...

2026-05-22 17:12:46 ERROR Couldn't parse Exec block at C:\Program Files\nxlog\conf\nxlog.conf:27; couldn't parse statement at line 27, character 33 in C:\Program Files\nxlog\conf\nxlog.conf; procedure 'to_syslog_ietf()' does not exist or takes different arguments

2026-05-22 17:12:46 ERROR module 'salida_syslog' has configuration errors, not adding to route 'ruta_logs' at C:\Program Files\nxlog\conf\nxlog.conf:32

2026-05-22 17:12:46 ERROR route ruta_logs is not functional without output modules, ignored at C:\Program Files\nxlog\conf\nxlog.conf:32

2026-05-22 17:12:46 WARNING no routes defined!

2026-05-22 17:12:46 WARNING not starting unused module entrada_eventos

2026-05-22 17:12:46 WARNING not starting unused module salida_syslog

2026-05-22 17:12:46 INFO nxlog-ce-3.2.2329 started

2026-05-22 22:57:01 WARNING stopping nxlog service

2026-05-22 22:57:01 WARNING nxlog-ce received a termination request signal, exiting...

2026-05-22 22:57:02 ERROR Couldn't parse Exec block at C:\Program Files\nxlog\conf\nxlog.conf:27; couldn't parse statement at line 27, character 32 in C:\Program Files\nxlog\conf\nxlog.conf; procedure 'to_syslog_bsd()' does not exist or takes different arguments

2026-05-22 22:57:02 ERROR module 'salida_syslog' has configuration errors, not adding to route 'ruta_logs' at C:\Program Files\nxlog\conf\nxlog.conf:32

2026-05-22 22:57:02 ERROR route ruta_logs is not functional without output modules, ignored at C:\Program Files\nxlog\conf\nxlog.conf:32

2026-05-22 22:57:02 WARNING no routes defined!

2026-05-22 22:57:02 WARNING not starting unused module entrada_eventos

2026-05-22 22:57:02 WARNING not starting unused module salida_syslog

2026-05-22 22:57:02 INFO nxlog-ce-3.2.2329 started

2026-05-22 23:38:03 WARNING received a system shutdown request

2026-05-22 23:38:03 WARNING stopping nxlog service

2026-05-22 23:38:03 WARNING nxlog-ce received a termination request signal, exiting...

2026-06-11 18:29:34 ERROR Couldn't parse Exec block at C:\Program Files\nxlog\conf\nxlog.conf:27; couldn't parse statement at line 27, character 32 in C:\Program Files\nxlog\conf\nxlog.conf; procedure 'to_syslog_bsd()' does not exist or takes different arguments

2026-06-11 18:29:34 ERROR module 'salida_syslog' has configuration errors, not adding to route 'ruta_logs' at C:\Program Files\nxlog\conf\nxlog.conf:32

2026-06-11 18:29:34 ERROR route ruta_logs is not functional without output modules, ignored at C:\Program Files\nxlog\conf\nxlog.conf:32

2026-06-11 18:29:34 WARNING no routes defined!

2026-06-11 18:29:34 WARNING not starting unused module entrada_eventos

2026-06-11 18:29:34 WARNING not starting unused module salida_syslog

2026-06-11 18:29:34 INFO nxlog-ce-3.2.2329 started

Gemini ha dicho

¡Espectacular! Has encontrado el santo grial del problema. El registro `nxlog.log` nos está cantando el error exacto y con todo lujo de detalles:

> `ERROR procedure 'to_syslog_bsd()' does not exist or takes different arguments WARNING no routes defined! (not starting unused module salida_syslog)`

¿Qué está ocurriendo?

NXLog se está negando en redondo a arrancar sus motores porque **le faltan los módulos de conversión a Syslog al principio del archivo**. Al no entender qué significa `to_syslog_bsd()` (o `to_syslog_ietf()`), el programa anula por completo la salida y deshabilita la ruta de envío. Por eso tu servidor Ubuntu jamás recibe nada.

Para que esas funciones existan, **tienes que cargar obligatoriamente las extensiones `xm_syslog` al inicio del archivo**.

La Solución: Modificar el `nxlog.conf` en Windows

Abre tu Bloc de notas en Windows **como Administrador**, ve a `C:\Program Files\nxlog\conf\nxlog.conf` y reemplaza por completo el contenido por este código corregido.

He añadido la sección `#### EXTENSIONES ####` arriba del todo, que es justo el ingrediente secreto que te faltaba:

Plaintext

```
define ROOT C:\Program Files\nxlog

Moduledir %ROOT%\modules
CacheDir  %ROOT%\data
Pidfile   %ROOT%\data\nxlog.pid
SpoolDir  %ROOT%\data
LogFile   %ROOT%\data\nxlog.log

##############################################################################
#### EXTENSIONES (Obligatorio para que funcionen las macros de Syslog) ####
##############################################################################
<Extension syslog>
    Module      xm_syslog
</Extension>

#################################
#### MÓDULOS DE ENTRADA      ####
#################################
# Lee el Visor de Eventos de Windows
<Input entrada_eventos>
    Module      im_msvistalog
    Query       <QueryList>\
                    <Query Id="0">\
                        <Select Path="Application">*</Select>\
                        <Select Path="System">*</Select>\
                        <Select Path="Security">*</Select>\
                    </Query>\
                </QueryList>
</Input>

#################################
#### MÓDULOS DE SALIDA       ####
#################################
# Convierte y envía los datos en formato Syslog estándar (UDP)
<Output salida_syslog>
    Module      om_udp
    Host        10.2.7.99
    Port        514
    Exec        to_syslog_bsd();
</Output>

#################################
#### RUTAS DE CONEXIÓN       ####
#################################
```

```
<Route ruta_logs>
    Path        entrada_eventos => salida_syslog
</Route>
```

Siguientes pasos obligatorios:

1. **Guarda el archivo** en el Bloc de Notas.
2. Abre la aplicación **Servicios** de Windows (`services.msc`).
3. Busca el servicio **nxlog**, haz clic derecho y selecciona **Reiniciar**.
4. Vuelve a abrir tu archivo de registro `C:\Program Files\nxlog\data\nxlog.log` para confirmar que ya no aparecen líneas con la palabra `ERROR`. Debería salir una línea limpia diciendo `INFO nxlog-ce-3.2.2329 started`.

¡En cuanto hagas esto, Windows empezará a escupir los eventos a la red y verás aparecer la ansiada carpeta de la IP en tu comando `ls -l /var/log/remote/` del Ubuntu!

Has dicho
2026-05-22 17:09:21 WARNING no functional input modules!

2026-05-22 17:09:21 WARNING no routes defined!

2026-05-22 17:09:21 INFO nxlog-ce-3.2.2329 started

2026-05-22 17:12:43 WARNING stopping nxlog service

2026-05-22 17:12:43 WARNING nxlog-ce received a termination request signal, exiting...

2026-05-22 17:12:46 ERROR Couldn't parse Exec block at C:\Program Files\nxlog\conf\nxlog.conf:27; couldn't parse statement at line 27, character 33 in C:\Program Files\nxlog\conf\nxlog.conf; procedure 'to_syslog_ietf()' does not exist or takes different arguments

2026-05-22 17:12:46 ERROR module 'salida_syslog' has configuration errors, not adding to route 'ruta_logs' at C:\Program Files\nxlog\conf\nxlog.conf:32

2026-05-22 17:12:46 ERROR route ruta_logs is not functional without output modules, ignored at C:\Program Files\nxlog\conf\nxlog.conf:32

2026-05-22 17:12:46 WARNING no routes defined!

2026-05-22 17:12:46 WARNING not starting unused module entrada_eventos

2026-05-22 17:12:46 WARNING not starting unused module salida_syslog

2026-05-22 17:12:46 INFO nxlog-ce-3.2.2329 started

2026-05-22 22:57:01 WARNING stopping nxlog service

2026-05-22 22:57:01 WARNING nxlog-ce received a termination request signal, exiting...

2026-05-22 22:57:02 ERROR Couldn't parse Exec block at C:\Program Files\nxlog\conf\nxlog.conf:27; couldn't parse statement at line 27, character 32 in C:\Program Files\nxlog\conf\nxlog.conf; procedure 'to_syslog_bsd()' does not exist or takes different arguments

2026-05-22 22:57:02 ERROR module 'salida_syslog' has configuration errors, not adding to route 'ruta_logs' at C:\Program Files\nxlog\conf\nxlog.conf:32

2026-05-22 22:57:02 ERROR route ruta_logs is not functional without output modules, ignored at C:\Program Files\nxlog\conf\nxlog.conf:32

2026-05-22 22:57:02 WARNING no routes defined!

2026-05-22 22:57:02 WARNING not starting unused module entrada_eventos

2026-05-22 22:57:02 WARNING not starting unused module salida_syslog

2026-05-22 22:57:02 INFO nxlog-ce-3.2.2329 started

2026-05-22 23:38:03 WARNING received a system shutdown request

2026-05-22 23:38:03 WARNING stopping nxlog service

2026-05-22 23:38:03 WARNING nxlog-ce received a termination request signal, exiting...

2026-06-11 18:29:34 ERROR Couldn't parse Exec block at C:\Program Files\nxlog\conf\nxlog.conf:27; couldn't parse statement at line 27, character 32 in C:\Program Files\nxlog\conf\nxlog.conf; procedure 'to_syslog_bsd()' does not exist or takes different arguments

2026-06-11 18:29:34 ERROR module 'salida_syslog' has configuration errors, not adding to route 'ruta_logs' at C:\Program Files\nxlog\conf\nxlog.conf:32

2026-06-11 18:29:34 ERROR route ruta_logs is not functional without output modules, ignored at C:\Program Files\nxlog\conf\nxlog.conf:32

2026-06-11 18:29:34 WARNING no routes defined!

2026-06-11 18:29:34 WARNING not starting unused module entrada_eventos

2026-06-11 18:29:34 WARNING not starting unused module salida_syslog

2026-06-11 18:29:34 INFO nxlog-ce-3.2.2329 started

2026-06-11 19:35:15 WARNING stopping nxlog service

2026-06-11 19:35:15 WARNING nxlog-ce received a termination request signal, exiting...

2026-06-11 19:35:18 INFO nxlog-ce-3.2.2329 started

[root@server2asir remote]$ls -l

total 20

drwxr-xr-x 2 syslog syslog 4096 may 22 19:24 10.2.7.3

drwx------ 2 syslog syslog 4096 may 23 11:02 2asircristobal

drwx------ 2 syslog syslog 4096 may 22 19:05 server2asir

drwx------ 2 syslog syslog 4096 may 23 11:06 usuario-Standard-PC-i440FX-PIIX-1996

drwx------ 2 syslog syslog 4096 jun 11 17:35 WIN-Q9RNRM3S3QT.empresa.local

Tu Nombre jueves 11 junio 2026 17:35

Gemini ha dicho

¡Brillante! Fíjate en la última línea de tu `ls -l`:

> `drwx------ 2 syslog syslog 4096 jun 11 17:35 WIN-Q9RNRM3S3QT.empresa.local`

¡Ahí la tienes! La carpeta acaba de aparecer mágicamente con el nombre DNS/Hostname de tu máquina Windows (`WIN-Q9RNRM3S3QT.empresa.local`).

Esto significa dos cosas:

1. **NXLog** ya ha arrancado sin errores, ha procesado el Visor de Eventos y está transmitiendo con éxito.
2. **Rsyslog** en tu Ubuntu Server (`.99`) ha recibido el paquete, ha saltado tus filtros de alertas porque era un log normal y ha ejecutado limpiamente la **Regla 3**, creando la estructura que querías.

Un último detalle estético (Por si prefieres la IP en vez del Nombre)

Si te das cuenta, las otras carpetas de tu servidor se guardan con la IP (`10.2.7.3`) y esta se ha guardado con el nombre de la máquina. Esto pasa porque en tu `rsyslog.conf` pusiste esto en la Regla 3:

Plaintext

```
$template DynamicFiles,"/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log"
```

Como Windows se ha identificado a sí mismo enviando su nombre de dominio completo (`WIN-Q9RNRM3S3QT.empresa.local`), Rsyslog ha usado el valor `%HOSTNAME%`.

Si en el futuro prefieres obligar a que **todas** las carpetas remotas se organicen estrictamente por su dirección IP (para que quede homogéneo con el `10.2.7.3`), solo tendrías que cambiar esa variable por `%FROMHOST-IP%`. Quedaría así:

Plaintext

```
$template DynamicFiles,"/var/log/remote/%FROMHOST-IP%/%PROGRAMNAME%.log"
```

Pero vaya, tal y como está ahora mismo, **la práctica está completamente terminada, funcional y recogiendo tanto métricas en Grafana como logs en Rsyslog**. ¡Buen trabajo cazando esos errores de sintaxis!

Has dicho
y para que las alertas críticas salgan en [root@server2asir remote]$pwd

/var/log/remote

Tu Nombre jueves 11 junio 2026 17:53

[root@server2asir remote]$ls -l ../ALERTA_TOTAL.log

-rw-rw-r-- 1 syslog adm 12421 may 23 11:20 ../ALERTA_TOTAL.log

Tu Nombre jueves 11 junio 2026 17:53

[root@server2asir remote]$?

Gemini ha dicho

Para conseguir que las alertas críticas de tu máquina Windows (y de cualquier otra máquina remota) **también dejen huella dentro del archivo `/var/log/ALERTA_TOTAL.log`** sin romper la creación de las carpetas dinámicas, el truco está en cómo gestionas la palabra clave **`stop`** en tu archivo de configuración.

El problema actual es que si una alerta remota entra y cumple los requisitos de la Regla 1, el comando `stop` hace que Rsyslog deje de procesar ese log inmediatamente, impidiendo que baje hasta la Regla 3 (la que crea la carpeta dinámica).

Para solucionarlo de forma limpia y mantener tu estructura, tienes dos opciones muy sencillas. Elige la que te resulte más cómoda:

Opción A: Quitar el `stop` de la Regla 1 (Recomendado)

Si quitas el `stop` de la regla de alertas, cuando llegue un log crítico ocurrirá lo siguiente:

1. Se guardará una copia en `/var/log/ALERTA_TOTAL.log`.
2. Como **no** hay un `stop`, el log seguirá bajando por el archivo.
3. Llegará a la Regla 3, verá que es remoto y creará/guardará el log en su carpeta correspondiente dentro de `/var/log/remote/`.

Abre tu archivo:

Bash

```
nano /etc/rsyslog.conf
```

Busca tu **Regla 1** (líneas 37-40 aproximadamente) y déjala exactamente así (simplemente comentando o borrando la línea del `stop`):

Plaintext

```
# 1. FILTRO DE ALERTAS TOTALES (Aquí interceptamos TODO lo grave de todos los
equipos)
if ($syslogseverity <= 3 or $msg contains "level=ERROR" or $msg contains
"GRAFANA_TEST" or $programname contains "critical" or $msg contains "critical") then
{
    action(type="omfile" file="/var/log/ALERTA_TOTAL.log")
    # stop   <-- COMENTAMOS O BORRAMOS ESTE STOP
}
```

Opción B: Duplicar la alerta dentro de la regla dinámica

Si por motivos de rendimiento o de diseño general de tu servidor necesitas obligatoriamente que la Regla 1 mantenga su `stop` para los logs locales del propio Ubuntu, puedes dejar la Regla 1 como está y añadir un "espejo" dentro de la Regla 3 para los equipos de la red.

En tu `nano /etc/rsyslog.conf`, localiza la **Regla 3** (abajo del todo) y modifícala para que quede así:

Plaintext

```
# 3. Todo lo demás que sea remoto va a las carpetas dinámicas
if $fromhost-ip != '127.0.0.1' then {

    # Si además de ser remoto es crítico, guardamos una copia en ALERTA_TOTAL
    if ($syslogseverity <= 3 or $msg contains "level=ERROR" or $msg contains
"GRAFANA_TEST" or $programname contains "critical" or $msg contains "critical") then
{
        action(type="omfile" file="/var/log/ALERTA_TOTAL.log")
    }

    # Continuamos guardándolo en su carpeta dinámica habitual
    $template DynamicFiles,"/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log"
    *.* ?DynamicFiles
    stop
}
```

Paso final obligatorio: Reiniciar para aplicar

Elijas la opción que elijas, una vez guardado el archivo con `Ctrl+O` y cerrado con `Ctrl+X`, reinicia el servicio del sistema operativo para que lea la nueva lógica de desvío:

Bash

```
systemctl restart rsyslog
```

¿Cómo probar que funciona?

Para comprobar que ahora sí que se duplica el log correctamente en ambos sitios:

1. Ve a tu máquina Windows, abre PowerShell como Administrador e inyecta un error crítico artificial:

   PowerShell

   ```
   Write-EventLog -LogName System -Source "Service Control Manager" -EventID 9999
   -EntryType Error -Message "CRITICAL_TEST: Alerta de prueba para comprobar el
   archivo total y carpetas"
   ```

2. Vuelve a tu Ubuntu Server y haz un `tail` al archivo general:

   Bash

   ```
   tail -n 10 /var/log/ALERTA_TOTAL.log
   ```

   (Deberías ver la línea del CRITICAL_TEST con la identidad de tu máquina de Windows).

3. Haz un `ls` o un `tail` a la carpeta del Windows para ver que también se ha guardado allí su copia local:

   Bash

   ```
   tail -n 10 /var/log/remote/WIN-Q9RNRM3S3QT.empresa.local/*.log
   ```

Has dicho
Funciona

Gemini ha dicho

¡Qué gran satisfacción da ver que todo encaja y funciona como un reloj suizo!

Haciendo un repaso de lo que has conseguido solucionar hoy (que no es poco para una arquitectura de monitorización real):

1. **En las métricas (Grafana + Prometheus):** Cazaste un error de espacios en el `prometheus.yml` que tenía tumbado el contenedor. Al alinearlo, Prometheus revivió y Grafana recuperó la vista al instante.
2. **En los logs de Windows (NXLog):** Corregiste la falta del módulo de extensión `xm_syslog` que hacía que el servicio de Windows se negara a arrancar. Al ponerlo, Windows empezó a transmitir.
3. **En el servidor de Logs (Rsyslog):** Ajustaste la lógica del flujo eliminando (o adaptando) el `stop` de la regla de alertas. Gracias a eso, ahora el servidor es capaz de duplicar la información en tiempo real: mete el aviso en el contenedor global de alertas (`ALERTA_TOTAL.log`) y, a la vez, clasifica el log limpiamente en su carpeta correspondiente dentro de `/var/log/remote/`.

Tienes montado un entorno de monitorización centralizado y securizado de nivel profesional. ¡Excelente trabajo de diagnóstico y resolución de problemas de sistemas!

Si necesitas cualquier otra cosa con los paneles de Grafana, el filtrado avanzado de logs o el resto de agentes (como el de Ubuntu o Mikrotik), por aquí me tienes. ¡A disfrutar del éxito de la práctica!

Has dicho
ahora, para el resto de equipos. necesito los comandos de comprobación, igual que para el Windows

Gemini ha dicho

Para que tengas la **guía de verificación completa** de cara a la entrega o defensa de la práctica, aquí tienes los comandos exactos de comprobación para el resto de los equipos de tu infraestructura (`fog-server`, `ubuntu-client` y `mikrotik`).

Con estos comandos podrás demostrar en segundos que cada sistema está vivo, transmitiendo métricas a Prometheus y enviando sus registros a Rsyslog.

1. Servidor FOG Linux (`10.2.7.5`)

Este servidor es el más completo porque monitoriza tres componentes distintos (Sistema, MariaDB y Apache).

Comprobación de Métricas (Prometheus/Grafana)

En tu servidor Ubuntu (`.99`), puedes comprobar si los tres exportadores están emitiendo datos lanzando peticiones web con `curl`:

Bash

```
# Verificar el Node Exporter (Hardware/Sistema - Puerto 9100)
curl -s http://10.2.7.5:9100/metrics | grep -m 1 "node_"

# Verificar el MySQL Exporter (Base de datos MariaDB - Puerto 9104)
curl -s http://10.2.7.5:9104/metrics | grep -m 1 "mysql_"

# Verificar el Apache Exporter (Servidor Web - Puerto 9117)
curl -s http://10.2.7.5:9117/metrics | grep -m 1 "apache_"
```

Si responden líneas con texto técnico y números, Grafana está recibiendo las métricas perfectamente.

Comprobación de Logs (Rsyslog)

En el servidor FOG, el agente encargado de enviar los logs suele ser el propio `rsyslog` nativo de Linux. Para forzar un log crítico de prueba desde FOG, ejecuta en su terminal:

Bash

```
logger -p local0.err "FOG_TEST_ERROR: Error crítico simulado en el servidor FOG"
```

Para verificar que ha llegado a tu servidor central (`.99`), ejecuta en la carpeta de logs:

Bash

```
# Ver si se ha creado su carpeta dinámica (buscará por su Hostname, ej: fogserver)
ls -l /var/log/remote/

# Ver si ha entrado en el contenedor global de alertas
grep "FOG_TEST" /var/log/ALERTA_TOTAL.log
```

2. Cliente Ubuntu (`10.2.7.127`)

Este equipo monitoriza su estado general de salud mediante `node_exporter`.

Comprobación de Métricas (Prometheus/Grafana)

Lanza un test desde tu servidor central (`.99`) hacia el puerto del cliente para ver si expone el hardware:

Bash

```
curl -s http://10.2.7.127:9100/metrics | grep -m 1 "node_"
```

Comprobación de Logs (Rsyslog)

Al igual que con FOG, puedes meter un log falso en el motor de logs del cliente Ubuntu para comprobar el desvío por red. Ejecuta en la terminal del **Cliente Ubuntu**:

Bash

```
logger -p auth.crit "UBUNTU_CLIENT_CRITICAL: Intento de acceso no autorizado
simulado"
```

Y compruébalo inmediatamente en tu **Servidor Central (`.99`)**:

Bash

```
# Comprobar si se ha creado su carpeta en el árbol remoto
ls -l /var/log/remote/

# Comprobar si ha saltado el filtro de alertas graves
grep "UBUNTU_CLIENT" /var/log/ALERTA_TOTAL.log
```

3. Router MikroTik

El MikroTik tiene una particularidad en tu escenario: sus métricas se recogen de forma indirecta mediante un contenedor Docker puente (`mikrotik-exporter`).

Comprobación de Métricas (Prometheus/Grafana)

Como configuraste en tu línea 32 del `prometheus.yml` que Prometheus apunte internamente al contenedor `mikrotik-exporter:9090`, puedes verificar si el exportador está traduciendo los datos del router haciendo un `curl` al puerto externo de ese contenedor (el puerto mapeado en tu `docker ps` es el `9094`):

Bash

```
# Ejecutar en tu servidor central (.99)
curl -s http://localhost:9094/metrics | grep -m 1 "mikrotik_"
```

Nota: Si te aparece `Up 27 minutes (unhealthy)` en el contenedor del MikroTik, revisa dentro de la configuración del script del contenedor si las credenciales de la API de MikroTik o la IP del router son las correctas.

Comprobación de Logs (Syslog del Router)

Los routers MikroTik no usan comandos de consola Linux como `logger`. Envían logs de forma constante por red (conexiones de firewall, inicios de sesión, cambios de interfaz) si has activado la acción `remote` en su menú `/system logging`.

Para verificar en tiempo real si el MikroTik está escupiendo logs hacia tu servidor central (`.99`), ejecuta este comando en la terminal de Ubuntu:

Bash

```
tcpdump -i any udp port 514 and host <IP_REAL_DEL_MIKROTIK> -nn -v
```

En cuanto el router registre cualquier movimiento, verás los paquetes impactar en tu pantalla. Al entrar al servidor, Rsyslog creará su respectiva carpeta en `/var/log/remote/<IP_o_Nombre_Mikrotik>/`.

💡 Resumen de comandos clave en el Servidor Central (`.99`) para la entrega:

- **Para ver el estado de salud global de la red de logs:** `ls -l /var/log/remote/` (Deben salir las carpetas de todos los equipos).
- **Para auditar incidentes graves de cualquier máquina:** `tail -n 20 /var/log/ALERTA_TOTAL.log`
- **Para verificar que todos los objetivos de Prometheus están en verde:** Abre en un navegador `http://10.2.7.99:9090/targets` y confirma que todos los *Jobs* (del 1 al 5) marquen estado **`UP`**.

Has dicho
que ocurre si hay más de un equipo Windows?

Gemini ha dicho

Si en tu red de monitorización añades más equipos Windows (por ejemplo, `Windows-Server-2`, `PC-Desarrollo-1`, etc.), la infraestructura que tienes montada **está perfectamente preparada para escalarse**.

Esto es lo que ocurre y cómo se gestiona en cada componente:

1. En Prometheus (Métricas)

Prometheus necesita saber las direcciones IP de los nuevos equipos. Tienes dos opciones para añadirlos en tu archivo `prometheus.yml`:

- **Opción A (Todos juntos en el mismo Job):** Si quieres que todos formen parte del grupo `windows-exporter`, simplemente añade sus IPs separadas por comas dentro de los corchetes.

  YAML

  ```
  - job_name: 'windows-exporter'
    static_configs:
      - targets: ['10.2.7.66:9182', '10.2.7.67:9182', '10.2.7.68:9182']
  ```

- **Opción B (Separados por Jobs si son departamentos distintos):**

  YAML

  ```
  - job_name: 'windows-servidores'
    static_configs:
      - targets: ['10.2.7.66:9182']

  - job_name: 'windows-clientes'
    static_configs:
      - targets: ['10.2.7.67:9182']
  ```

2. En Grafana (Visualización)

¡Aquí viene la magia! No necesitas importar un dashboard nuevo para cada máquina Windows.

Los dashboards de Grafana (como el `14542` o el `24390`) están diseñados con **variables dinámicas**. En la esquina superior izquierda del mapa te aparecerá un menú desplegable llamado **"Host"** o **"Instance"**.

- Al hacer clic en él, se desplegará la lista con todas las IPs de tus Windows (`10.2.7.66`, `10.2.7.67`, etc.).
- Al seleccionar uno, todas las gráficas de CPU, RAM y disco cambiarán en tiempo real para mostrar los datos exclusivos de esa máquina.

3. En Rsyslog (Logs del Servidor Central)

Gracias a la **Regla 3** con plantilla dinámica (`%HOSTNAME%`) que revisamos y corregimos en tu archivo `/etc/rsyslog.conf`:

Plaintext

```
$template DynamicFiles,"/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log"
```

No tienes que tocar absolutamente nada en el servidor Ubuntu. En cuanto configures `NXLog` en el segundo o tercer Windows apuntando al Ubuntu (`10.2.7.99`), Rsyslog detectará automáticamente el nombre de la nueva máquina (o su IP) y **creará una carpeta nueva de forma autónoma** en `/var/log/remote/`.

Por ejemplo, al hacer `ls -l` verías:

Plaintext

```
drwx------ 2 syslog syslog 4096 jun 11 17:35 WIN-Q9RNRM3S3QT.empresa.local
drwx------ 2 syslog syslog 4096 jun 12 11:15 WIN-SEGUNDOSERVER.empresa.local
```

4. En el archivo `ALERTA_TOTAL.log`

Si cualquiera de los nuevos equipos Windows sufre un error crítico (un servicio que se cae, un disco lleno, etc.), su alerta viajará al servidor central y se guardará en `/var/log/ALERTA_TOTAL.log`.

En el texto del log verás reflejado qué equipo ha sido el que ha fallado, ya que el propio mensaje de Syslog incluye el Hostname o la IP de la máquina de origen, permitiéndote identificar cuál de todos los Windows está protestando.

